

RAG Chatbot Capstone Project

System Architecture & Autonomous Antigravity Execution Goal Prompt

Document Version: 1.0.0	Engine: Google Antigravity	Artifact: Workpack Specification
Domain: Generative AI & NLP	Vector DB: ChromaDB / FAISS	Frameworks: LangChain, FastAPI, Streamlit

1. PROJECT OVERVIEW & ARCHITECTURE SPECIFICATIONS

The **RAG Chatbot Capstone Project** builds an enterprise-grade Retrieval-Augmented Generation system. It processes custom document collections (PDFs, Markdown, TXT), performs semantic chunking, stores vector embeddings in ChromaDB, retrieves relevant context using similarity search, and streams factual responses through an LLM via a FastAPI backend and Streamlit UI.

Component	Technology Stack	Description
Ingestion & Chunking	PyPDF + RecursiveCharacterTextSplitter	Extracts document text and breaks it down into semantic chunks with token overlaps.
Vector Database	ChromaDB / FAISS + HuggingFace/ OpenAI Embeddings	Indexes dense embedding vectors for similarity retrieval with metadata filtering.
Retrieval & Orchestration	LangChain LCEL / LlamaIndex	Combines context retrieval, prompt engineering, and LLM text generation with source citations.
Web Backend & UI	FastAPI + Streamlit Dashboard	Exposes RESTful API endpoints (/upload , /query , /health) and interactive web chat.

2. ANTIGRAVITY GOAL EXECUTION PROMPT

Copy and paste the exact block below into Google Antigravity in `/goal` mode to autonomously construct the repository:

```
/goal Build a complete, production-ready RAG (Retrieval-Augmented Generation) Chatbot Capstone Project repository in Python using LangChain, ChromaDB, FastAPI, Streamlit, Docker, unit tests, and a GitHub-ready README.md.

### DIRECTORY STRUCTURE TO CREATE:
rag_chatbot_project/
├── data/
│   ├── raw_documents/      # Sample PDF, TXT, and MD files for ingestion
│   └── vector_store/        # Persisted ChromaDB collection files
├── src/
│   ├── __init__.py
│   ├── document_loader.py  # PDF/Text ingestion & RecursiveCharacterTextSplitter logic
│   └── vector_db.py         # ChromaDB initialization, embedding generation, and similarity
search
├── rag_chain.py             # LangChain LCEL retrieval chain with custom prompt templates &
citations
├── utils.py                 # Memory session handler & logger configuration
├── app/
│   ├── api.py              # FastAPI REST endpoints (POST /upload, POST /query, GET /health)
│   └── ui.py               # Interactive Streamlit chat interface with document uploader &
source inspector
├── tests/
└── test_ingestion.py       # Unit tests for text splitters & vector creation
```

```

├── test_api.py          # Pytest suite verifying FastAPI query routes
├── requirements.txt
├── Dockerfile
├── main.py              # CLI launcher script (--mode ingest|serve|test)
└── README.md            # Comprehensive, GitHub-ready README file

---

### SPECIFIC REQUIREMENTS FOR GENERATED README.md:
1. Badges: Python 3.10+, LangChain, ChromaDB, FastAPI, Streamlit, License MIT, Build Passing.
2. Project Overview: Complete technical walkthrough of RAG architecture, vector search, and hallucination reduction.
3. Architecture Diagram: Mermaid.js flowchart showing Document Ingestion -> Vector Embedding -> Query Retrieval -> Prompt Context Injection -> LLM Generation -> UI Stream.
4. Evaluation & Metrics: Benchmark table highlighting Retrieval Precision@K, Latency, and Context Relevance.
5. Quickstart Instructions: Environment setup, document ingestion CLI commands, local server launch, Streamlit deployment, and Docker container instructions.
6. API Specifications: Swagger/OpenAPI endpoint schemas (`/upload`, `/query`, `/health`) with sample request/response JSON payloads.

---

### WORKFLOW STEPS:
1. Implement `src/document_loader.py` for PDF and text ingestion with chunking.
2. Implement `src/vector_db.py` handling embedding creation and ChromaDB persistence.
3. Construct `src/rag_chain.py` integrating LangChain retrieval and prompt formatting.
4. Build FastAPI backend (`app/api.py`) exposing async document ingestion and query endpoints.
5. Create interactive Streamlit web dashboard (`app/ui.py`) with real-time response streaming and source citation toggles.
6. Build Dockerfile exposing ports 8000 and 8501.
7. Implement unit test suite in `tests/` verifying ingestion and query chains.
8. Generate the complete, production-grade GitHub `README.md`.
9. Execute `pytest` to confirm all test cases pass cleanly.

```